Hand Tracking Gesture Controlled Snake Game

Group member： Li Xiang、Hu Xingyu（code）

Wang Ruiyi、Yin Jie(report writing)

Pei Yifei、Yin Shaohan(PPT)

Abstract. This project implements a gesture-controlled Snake game based on computer vision and hand landmark detection. Using Python, OpenCV, and cvzone, the system captures real-time camera input, tracks the user's index finger tip, and uses it to control the movement of the snake. Core game mechanics including snake growth, food collection, score counting, self-collision detection, and game reset are fully realized. The project replaces traditional keyboard control with touchless gesture interaction, achieving smooth response and stable performance. This report describes the system design, implementation details, testing results, optimizations, and conclusions.

DIRECTORY

1.Introduction.....

.....3

1.1 Background

1.2Objectives

1.3 Significance

2. Technology Stack & Development Environment.....3

2.1 Core Technology Stack

2.2 Development Environment

2.3 Environment Setup

3. System Design.....4

3.1 Overall Architecture

3.2 Core Class Design

4. Core Function Implementation.....5

4.1 Camera & Hand Tracking Initialization

4.2 Snack Core Logic

4.2.1 Movement Control

4.2.2 Length Control

4.2.3 Food Control

4.2.4 Self-Collision Dection

4.3 UI Rendering

4.3.1Snack Body & Head

4.3.2 Food Overlay

4.3.3 Score & Prompts

4.4 Game Control

5. Game Execution Flow 7

5.1 Startup

5.2 Runtime

5.3 Shutdown

6. Testing & Result Analysis 8

6.1 Function Test

6.2 Performance Test

6.3 Issues & Soluctions

7. Project Optimization & Troubleshooting 9

7.1 Warning Suppression

7.2 Robustness Improvements

7.3 Future Improvements

8. Conclusion 9

1. Introduction

1.1 Background

Traditional Snake games rely on keyboard or button input, which limits interaction flexibility. With the development of human–computer interaction (HCI) and computer vision, gesture control has become a natural and intuitive way to interact with devices. This project combines hand tracking technology with the classic Snake game to explore non-contact control in entertainment applications.

1.2 Objectives

- To realize real-time hand detection and finger tracking.
- To design and implement a complete gesture-controlled Snake game.
- To achieve food eating, length extension, score system, and collision detection.
- To support game reset and quit functions.
- To ensure stable running and low control latency.

1.3 Significance

- Provides a novel interactive experience without external controllers.
- Demonstrates the application of computer vision in simple game development.
- Improves programming and team collaboration skills.

2. Technology Stack & Development Environment

2.1 Core Technology Stack

Technology/Library	Function
Python	Main programming language
OpenCV	Image capture, drawing, collision detection
cvzone	Hand tracking, PNG overlay, text rendering
NumPy	Array processing, collision calculation
math	Distance calculation
random	Random food position generation

Table 1. Project Technology Stack and Development Environment

This table shows the core programming languages, third-party libraries, and built-in modules used in the project. It lists the recommended version requirements and corresponding functions of each technology, providing a complete and stable environment foundation for realizing hand tracking, game logic operation, image processing and interface rendering.

2.2 Development Environment

- OS: Windows / Linux / macOS
- Hardware: Camera (built-in/external), basic CPU for image processing
- Tools: PyCharm / VS Code, pip

2.3 Environment Setup

- Install dependencies via pip:
- pip install opencv-python cvzone numpy

3. System Design

3.1 Overall Architecture

The project consists of three core modules:

Camera & Hand Detection Module: Initializes camera, captures frames, detects hand key points.

Game Logic Module: Encapsulates snake movement, length control, food generation, collision detection.

UI Rendering Module: Draws snake, food, score, and prompts.

3.2 Core Class Design

The core is the SnakeGameClass, which contains all game states and behaviors.

Type	Name	Description
Attribute	points	List of all body points
Attribute	lengths	Distances between adjacent points
Attribute	allowedLength	Maximum allowed length
Attribute	foodPoint	Food coordinates
Attribute	score	Current score
Attribute	gameOver	Game over flag
Method	Init	Initialize resources and food
Method	randomFoodLocation	Generate random food position
Method	update	Main update: movement, length, collision, rendering

Table 2. Core attributes and methods of SnakeGameClass

This table lists the main member variables and methods in the core class SnakeGameClass. Attributes are used to store the real-time state of the snake, food, score and game status. Methods are used to realize resource initialization, food generation and the whole game logic update, including movement control, length management, collision detection and interface rendering.

4.Core Function Implementation

4.1 Camera & Hand Tracking Initialization

Initialize camera and hand detector with fixed resolution and single-hand detection.

4.2 Snake Core Logic

4.2.1 Movement Control

The snake head follows the index finger tip.

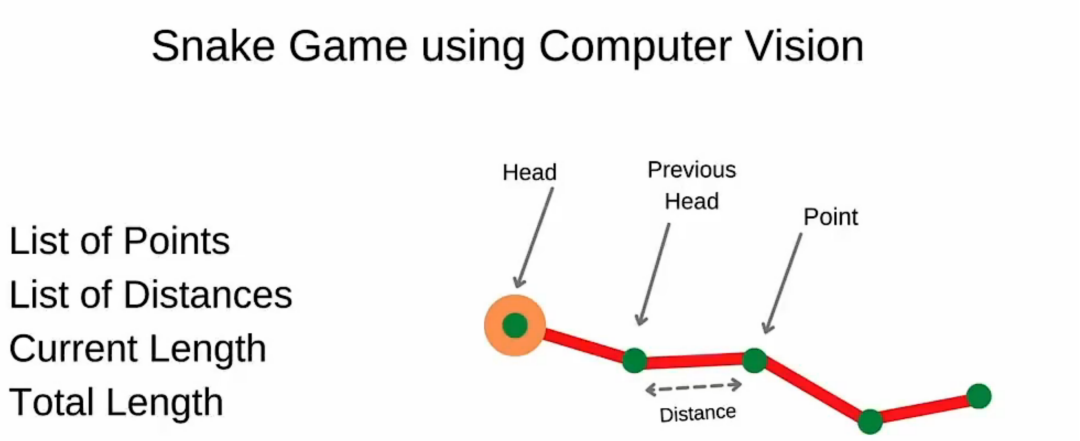


Figure 1. Schematic diagram of the snake body data structure

This figure illustrates the core data structure used to represent the snake in the game. The snake is modeled by a list of coordinate points (List of Points), a list of distances between adjacent points (List of Distances), the real-time total length of the snake (Current Length), and the maximum allowed length (Total Length). The head follows the user's fingertip, and the body is constructed by connecting consecutive points, ensuring smooth movement.

4.2.2 Length Control

Trim the tail when exceeding allowed length.

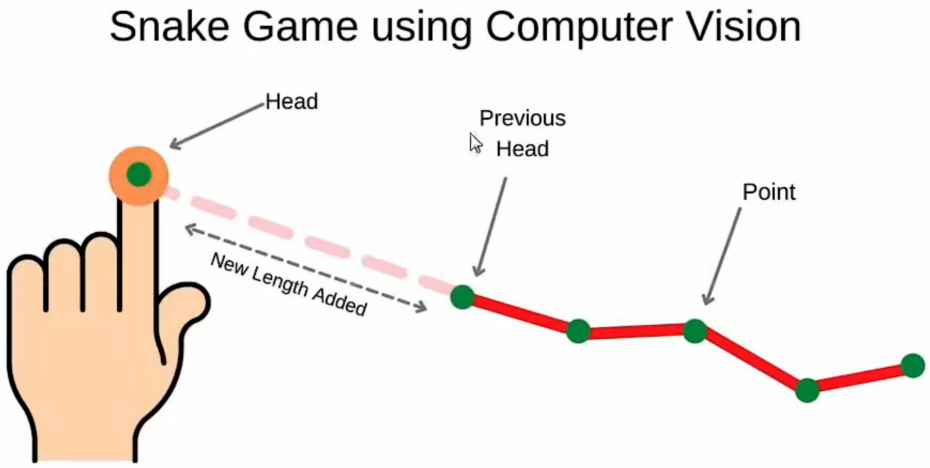


Figure 2. Schematic diagram of snake head following and length update

This figure shows how the snake moves and updates its length. When the user moves their index finger, a new head position is generated, and the distance between the new head and the previous head is calculated and added to the total length. The snake's body follows the head's movement in real time, realizing gesture-controlled movement.

4.2.3 Food Collision

Check if head overlaps food area; increase length and score if eaten.

4.2.4 Self-Collision Detection

Use pointPolygonTest to detect head–body collision.

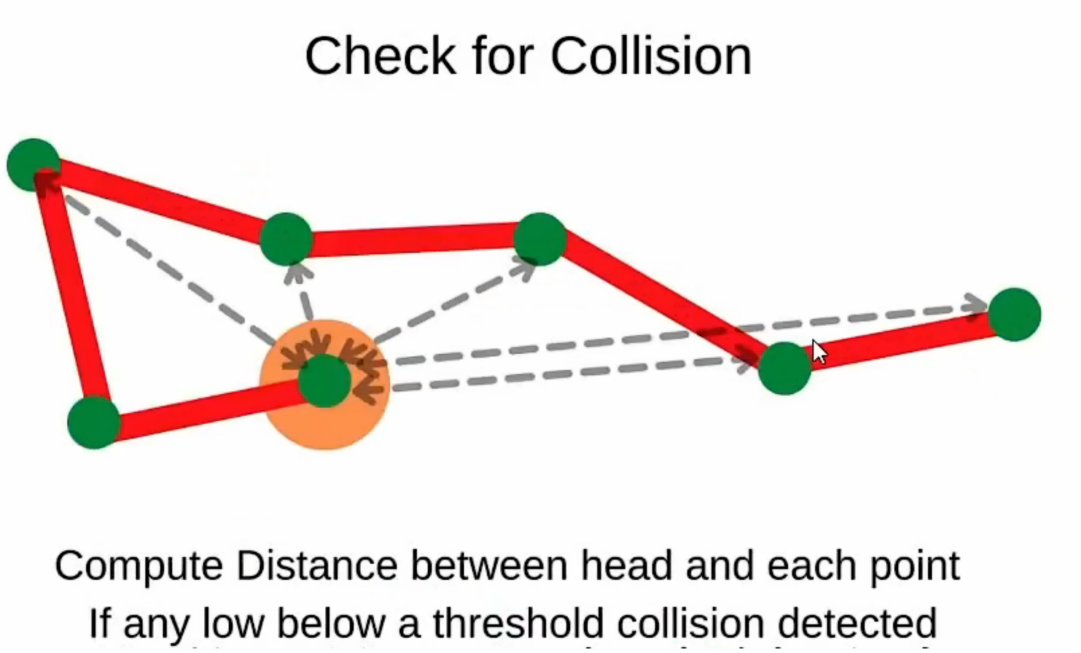


Figure 4. Schematic diagram of self-collision detection

This figure shows the collision detection logic of the game. The system calculates the distance between the snake head and each body point. If the distance is below a set threshold, a collision is detected, and the game ends. We use OpenCV's cv2.pointPolygonTest function to implement this detection efficiently.

4.3 UI Rendering

4.3.1 Snake Body & Head

Draw red body line and purple head circle.

4.3.2 Food Overlay

Overlay transparent PNG food image.

4.3.3 Score & Prompts

Display score and Game Over text.

4.4 Game Control

Support reset (R) and quit (Q).

5. Game Execution Flow

5.1 Startup

Initialize camera and detector.

Load food image and suppress warnings.

Create SnakeGameClass instance.

Enter main loop with flipped mirror frame.

5.2 Runtime

Detect hand and get index finger tip.

Call update() to refresh game state.

Check food collision and update score.

Check self–collision and trigger game over.

Render UI and handle key inputs.

5.3 Shutdown

Release camera.

Destroy all OpenCV windows.

Exit program.

6. Testing & Result Analysis

6.1 Function Test

Test Item	Steps	Expected Result	Actual Result
Hand tracking	Move hand in front of camera	Correctly detect index finger	Pass
Snake movement	Move finger	Snake follows smoothly	Pass
Eat food	Touch food	Score +1, length increases	Pass
Collision	Head touches body	Game Over	Pass
Reset	Press R	Game restarts	Pass
Quit	Press Q	Program exits	Pass

Table 3. Function Test Results

This table presents the functional test results of the gesture-controlled snake game. Six key functions were tested, including hand tracking, snake movement, food eating, collision detection, game reset and quit. All test items met the expected results, proving that the system is stable and all functions work correctly.

6.2 Performance Test

Frame rate: 25–30 FPS on regular laptops.

Latency: <100ms between gesture and response.

Compatibility: Runs on Windows 10/11 and Ubuntu 20.04.

6.3 Issues & Solutions

Issue	Cause	Solution
libpng warnings	PNG contains iCCP profile	Copy image + filter warnings
Food out of screen	Unbounded random coordinates	Limit to safe range
False collision	Too few body points	Detect only when len(points) > 5

Table 4. Issues, Causes and Solutions

This table summarizes the common problems encountered during project development, including libpng warnings, out-of-screen food generation and false collision detection. It provides clear causes and effective solutions for each issue to ensure the stability and reliability of the game system.

7. Project Optimization & Troubleshooting

7.1 Warning Suppression

7.2 Robustness Improvements

Exception handling for missing food image.

Convert coordinates to integers for drawing.

Unified method naming to avoid errors.

7.3 Future Improvements

Difficulty progression based on score.

Multi-hand support.

Sound effects for eating and collision.

Local high-score saving.

8. Conclusion

This project successfully designed and implemented a gesture-controlled Snake game based on hand tracking and computer vision. The system uses OpenCV and cvzone to capture real-time video and detect the index finger tip, realizing non-contact control of the snake movement. Core functions including snake following, length changing, food collection, score counting, self-collision detection, game reset and exit are all completed and run stably.

During the development, we solved problems such as library warnings, image loading and collision misjudgment, and improved the robustness and fluency of the program. The test results show that the system has low delay, smooth control and high compatibility, which achieves the expected design goals.

This project combines traditional games with computer vision technology, providing a new human-computer interaction experience. It also helps us master image processing, hand tracking and game logic development.